1 Front Matter

1.1 Collaboration Statement

The last page of the course syllabus contains a statement of the department collaboration policy with regard to projects. Violation of the collaboration policy can result in failure of the course.

1.2 Reminder: Project Hand in Rules

Read and follow the instructions!

1. All project due dates are specified on Brightspace.
2. All projects handed in must be NetBeans projects.
3. Each student starts the semester with 7 free late days. When handing in a project late you must add a comment telling me that you are using some number of late days for the project.
4. If you hand in a project late, you are not allowed to correct and resubmit. This is the only penalty for late submissions.
5. For projects, you will hand in what is specified by individual assignments. Most of the time it will be an entire zipped project folder. Name the archive with ONLY YOUR LAST NAME. Example: Smith.zip ONLY zip archives will be accepted.

1.3 Objectives

As your first project in this course many of the objectives are those things which you are expected to do for every project. Be sure to read and understand the grading criteria for each project.

1. Learn to read and follow directions!
2. The main() method

- Correct usage of command line arguments in main, and setting them in the project properties.
- Correct creation of an object to call the run() method

3. The run() method. Most other methods must be called from the run() method.
4. Correct reading of the file with all exceptions caught and handled.
5. Understanding how Selection Sort works.
6. Using formatted strings.
7. Using parallel arrays.
8. Calculating simple sums and percentages.

1.4 Grading Criteria

Each group of requirements below is graded as all or nothing. Grading for this project will be done as follows:

Grade Earned	Requirements
D	The main method is the only static method in the class. The main method checks the command line arguments. Usage message is correct for the number of required arguments. The main method in the driver creates an object of the class and calls a run() method passing the arguments. All code handed in is syntactically correct and correctly formatted. The program compiles and runs.
C	Everything above Plus: Correctly reads the data into arrays and checks for appropriate exceptions. Correctly sorts the transactions by category. Correctly calculates and displays subtotals for each category.
B	Everything above plus: Correctly re-sorts the transactions by check number. Correctly displays each transaction as a series of formatted Strings.
A	Everything above plus: Correctly displays the running balance along with each transaction. Correctly calculates the savings balance and a new balance forward at the end of the year.

2 Introduction

Knowing how to handle money and how to budget are necessary life skills. There are lots of apps to help you do that. SoFi, Mint, Quicken, and more will help you see where your money is going with nice charts and graphs. We are going to write our own personal finance app.

3 Specification

Write these things in this order. Don't move on until you have one part working perfectly! This is laid out this way to emphasize how specification grading works. You won't get this much detail on any other project. Remember to follow all of the coding conventions including the order of the methods.

1. For a grade of D:
   (a) Create a project in NetBeans called PersonalFinances. Using the instructions from the BookStore lab, put the data file from Brightspace in the right place in the project folder and put the Date.java file in the right place in the project folder
   (b) This project initially requires one command-line argument. Using the instructions from the BookStore lab, create a main method that checks for one command line argument. Test your program before adding the run-time argument to the project properties.
   (c) The main method should then create a PersonalFinances object and call the run method passing the args array. (eventually there will be a second parameter).
2. For a grade of C: Write 3 methods from scratch and adapt two given methods for use in this project.
   (a) Using the instructions in the input file format section, declare an appropriate array for each of the data elements in the file. These should be declared private like the the array is the BookStore lab.
   (b) Write a run() method that calls a method that reads the file, passing the first argument in the array.
   (c) Write a method to read the file into the 6 arrays that you declared as properties. Create the arrays with a length of 250 except for the column headings array which is size 6. Use the readFile() method from the BookStore lab as an example including all of the catch clauses. Don't forget to count the actual number of records in the file (declare it as a property). You should print the number of records read in and verify that with the data file. See the implementation notes.
   (d) Using the selectionSort() method included below, adjust it so that it sorts by category. Look at the compareTo() method in String to figure out how to do this. Name your method sortByCategory(). See the implementation notes below.
   (e) Using the swap() method also included below rewrite it so that it swaps the two elements in all 6 arrays. See the implementation notes below.
   (f) Write a method that will print the transactions sorted by category along with subtotals for each category.
   (g) Add calls to both methods to run().
3. For a grade of B: Write 2 new methods:

(a) Write a sort that sorts by check number. Since check number is an integer, the comparison will look like the original example code, but you still can't use the length of the array. you will use the same swap method that you already wrote.

(b) Write a method that will print the transactions using formatted Strings.

(c) Add calls to both methods to run().

4. For a grade of A:
   Rewrite and add to the existing methods.

   (a) Calculate the total debits and total credits in the print out by category and report it at the end of the output in that method.

   (b) Change the method that prints the transactions to accept a beginning balance for the checking account. Make that amount one of the command-line arguments. The example output uses $1,000.00 as a beginning balance. Adjust the command-line check in main() and the Usage message to match this requirement. Since it is being read in as a string you must convert it to a double to use it

```
Double.parseDouble(args[1])
```

   (c) Add a column to the output from the print method that prints a running balance after each transaction. Credits are added, debits are subtracted.

   (d) Pay yourself first! Before updating the balance, "deposit" 10% of your paycheck in your savings account and add the rest to the running balance.

   (e) At the end of the method move everything above the opening balance into savings and display both the savings balance and the new balance for the start of the next year.

4 Implementation Notes

Implementation Notes will be a text file in the project kit when I have something helpful to tell you. So read them, don't look for them in the project assignment again.

There is a video about formatted strings and an example of using a StringBuilder object to put a returned string together. Positive widths justify to the right, negative widths justify to the left. The width of '15' in the example below includes the decimal place and the 2 digits to the right of the decimal place. If the largest actual value is 'nnnn.nn' then there will be 8 empty columns to the left of that number. The spaces in the descriptor string are printed. If your stated widths are correct, you don't need the spaces shown in the string below. You must use this when I specify "a formatted string".

```
String eol = System.lineSeparator();
str.append(String.format("%7d %-30s %15.2f %s",
                         checkNum, descr, amount, eol));
```

Look at the BookStore Lab. Your main method should be very much like that one. You should have a run() method that accepts the array of arguments from main. To make it

easier to read in the individual integers and data values use the following command so that the slashes in the dates and the commas between the data values are all used as delimiters for reading. This statement needs to go before you start reading anything from the file.

```
inFile.useDelimiter("[/, \t\n\r]+");
```

As you read in descriptions and categories be sure to replace underscores with spaces. Look up the String method replaceAll() and read how to use it.

Here is the normal code for the selection sort. There are only a few things that you will need to change to use it to sort by the categories.

```
/**
  * Selection Sort finds the smallest element and puts it in first element
  * of the unsorted array. Decreases the size of the array to search at
  * each iteration.
  *
  * @param a the integer array
  */
public void selectionSort(int[] a) {

   int i, j, iMin;
   /* find the min element in the unsorted a[j .. n-1] */
   for (j = 0; j < a.length - 1; j++) {
      /* assume the min is the first element initially */
      iMin = j;
      /* test against elements after j to find the smallest */
      for (i = j + 1; i < a.length; i++) {
         /* if this element is less, then it is the new minimum */
         if (a[i] < a[iMin]) {
             /* hang on to that index */
             iMin = i;
         }
      }
      if (iMin != j) {
          swap(j, iMin);
      }
  }
}
```

Rename the method sortByCategory(). Don't pass an array to it, use the array that holds the categories. You can't use the array length, you must substitute the actual number of records where the length of the array is used. Since the elements we are comparing are Strings instead of integers you need to use the String class' compareTo() method instead of the simple comparison for integers. Google "Java api String" and read about the compareTo() method.

Rewrite the swap() method below so that it swaps elements in all 6 arrays (one array at a time).

```java
/**
  * Almost all sorts use a swap algorithm, so a swap method saves lots of
  * lines of code! This method accepts an array and the indeces of the
  * elements to be swapped and swaps them.
  *
  * @param list an integer array
  * @param i the index of one element to be moved
  * @param j the index of the other element to be moved
  */
public void swap(int[] list, int i, int j) {
   int temp = list[i];
   list[i] = list[j];
   list[j] = temp;
}
```

Again, don't pass an array to this method, just the two index values. Simply duplicate these 3 lines for each of the 6 arrays. You won't need to **declare** 6 different temp variables you only need 4. One int for the check number, one Date for the date, one double for the amount, and one String that you can reuse for the other arrays. But you will need all 3 statements for each of the 6 arrays. That is a total of 20 lines of code including the method header and the closing curly brace.

Your run method for this project will be quite short. It should look similar to this:

```java
public void run(String[] args) {
    readFile(args[0]);
    sortByCategory();
    System.out.println(categorySubTotals());
    sortByCheckNumber();
    System.out.println(transactions(Double.parseDouble(args[1])));
}
```

5 Input File Format

The input file is is in comma delimited format (csv) Here are the first few lines:

```
Check#,Date,Description,Amount,Type,Category
1669,3/15/2020,Comcast,122.64,debit,Entertainment
1705,5/15/2020,TechnoCompany,1730.77,credit,Paycheck
1725,6/19/2020,Lakeside_Market,27.03,debit,Gasoline
```

The first line should be read into an array (length 6) of Strings called columnHeadings or headings or something else appropriate.

As you read in descriptions and categories be sure to replace underscores with spaces. Look up the String method replaceAll().

```
strVar = inFile.next().replaceAll("_", " ");
```

6 Output Examples

```
Check#    Date       Description                  Amount
  // lots of lines missing here
  1795    10/15/2020 Toyota Financial             359.29
  1811    11/15/2020 Toyota Financial             359.29
  1830    12/15/2020 Toyota Financial             359.29
Subtotal for category Car Loan                         4,311.48
  // lots of lines missing here
Total Debits:   39,254.15  Total Credits:   45,000.02


Check#  Date       Description                 Amount  Type    Category      Balance Forward
                                                                                  1,000.00
  1623  1/1/2020   TechnoCompany              1,730.77 credit  Paycheck         2,557.69
  1624  1/1/2020   Johnson Property Mgmt      1,600.00 debit   Rent               957.69
  1625  1/2/2020   CMP                          164.57 debit   Utilities          793.12

    // lots of lines missing here
Savings Balance   Adj Checking Balance
    5,745.87             1,000.00
```

Adding commas to numbers that are output is easy. Just put a comma between the percent sign and the column width number like this: `%,15.2`

7 Complete the Documentation

Generate JavaDoc Documentation and complete it. Choose Tools - Analyze JavaDoc. In the window at the bottom click the check box for the driver and click the button that says Fix Selected. This will create part of the documentation. You must then go back through each file adding file descriptions, and completing the documentation for each method.

I actually read your documentation. This is easy. It's not a term paper and it is not creative writing. Pay attention to the video on documentation.

8 PROGRAMMING STYLE

Your programs will be evaluated for formatting, use of meaningful identifiers, and documentation.

Although no points are allocated for that on this assignment, somewhere between twenty and thirty percent of your grade will derive from the programming style on this and all future assignments. Note, the NetBeans environment can take care of formatting for you and it can generate skeleton JavaDoc which you MUST complete. Up to 30 points can be deducted for failure to adhere to the coding conventions, failure to correctly format your code, and failure to complete the documentation.

Formatting is a question of clicking two times. Choose Source ==>Format. It is the first thing I do when I open your source file. If nothing changes, you did it already. NetBeans does not automatically wrap your lines. You have to do that. Break long statements at appropriate places so that they don't go beyond the line on the right of the editor window.